



VOID

moodwave

미니 프로젝트 01

MOOD *WAVE

박해준

박소연

송동현

이수아



박해준

- 메인 페이지 구현
- 전반적인 프로젝트 세팅
- 프로젝트 총괄 및 지휘

박소연

- 날씨 추천 페이지 기능 구현
- Figma 디자인

송동현

- 로그인 연동 구현
- 좋아요 기능 구현
- 백엔드 세팅

이수아

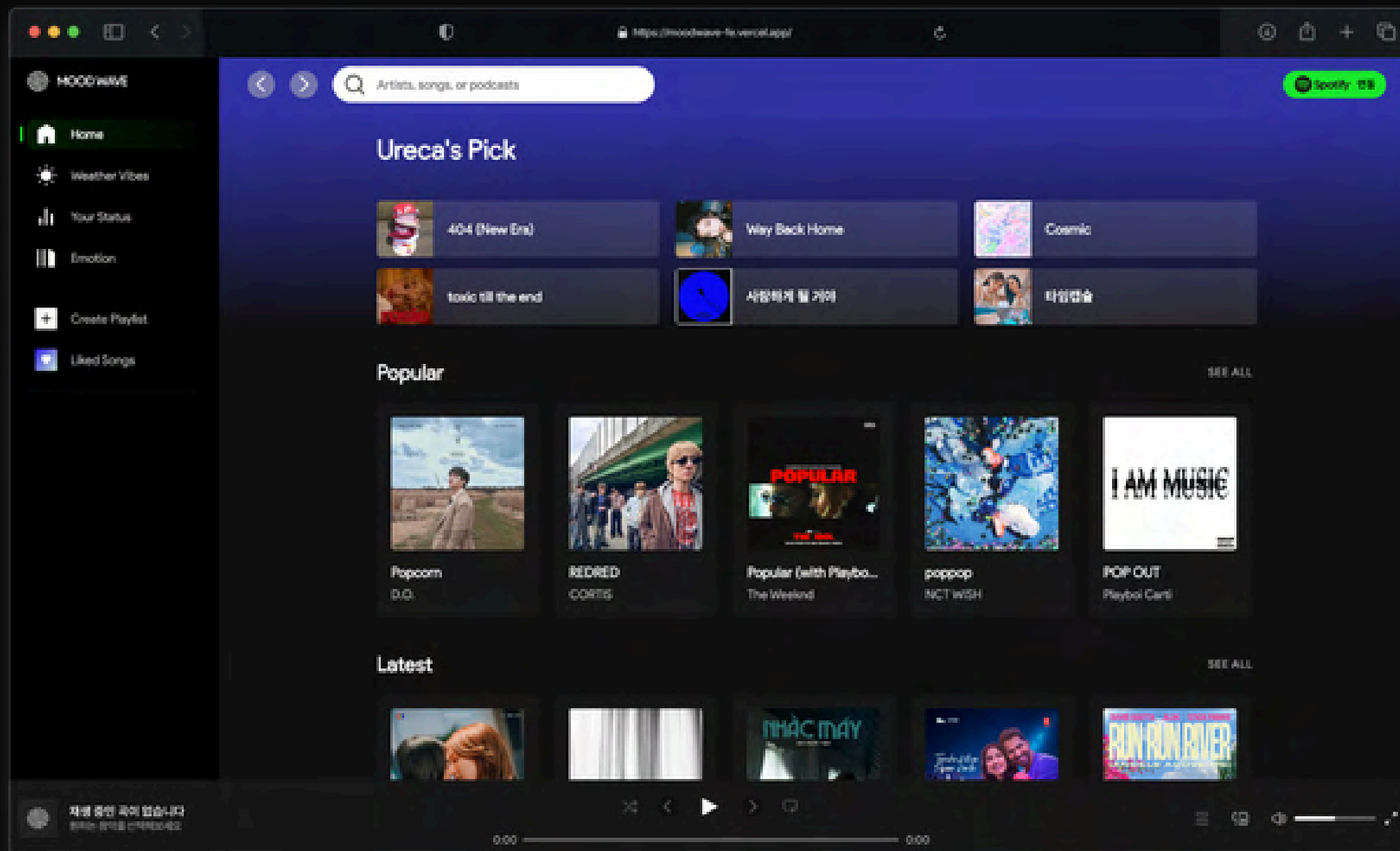
- 음악 취향 통계 페이지 구현
- Figma 디자인



MOOD WAVE

Spotify 기반 음악 스트리밍을
개인화 추천으로 차별화한 웹 서비스

- 01 실생활 활용성 강화
현재 날씨 기반 음악 추천
- 02 AI 감정 분석을 통한
추천 음악 생성
- 03 음악 취향 통계 시각화 기능



음악 검색 & 재생

좋아요 & 플레이리스트

날씨 기반 추천

음악 취향 통계

AI 감정 분석 추천



VERSION CONTROL



GitHub

Git Flow 전략

feature → dev → main 순으로
병합하여 안정적인 개발 및 배포를 진행



COMMUNICATION



Notion

기획 · 개발 문서
작성 및 관리



Figma

UI 디자인 및
화면 구조 공유



Slack

팀원 간 소통 및
진행 상황 공유

DEVELOPMENT TOOLS



VS code

프론트엔드
개발

Frontend



Eclipse

백엔드
개발

Backend



MySQL

DB 설계 및
관리

Database



Postman

API 테스트 및
검증

API Test

DEPLOYMENT



Vercel

프론트엔드
배포

Frontend



Railway

백엔드 및
DB 배포

Backend / DB



Frontend



Vite



Html5



CSS



JS

API



Spotify



OpenAI



OpenWeather

Backend



Spring Boot

Library



Chart.js

Database



MySQL

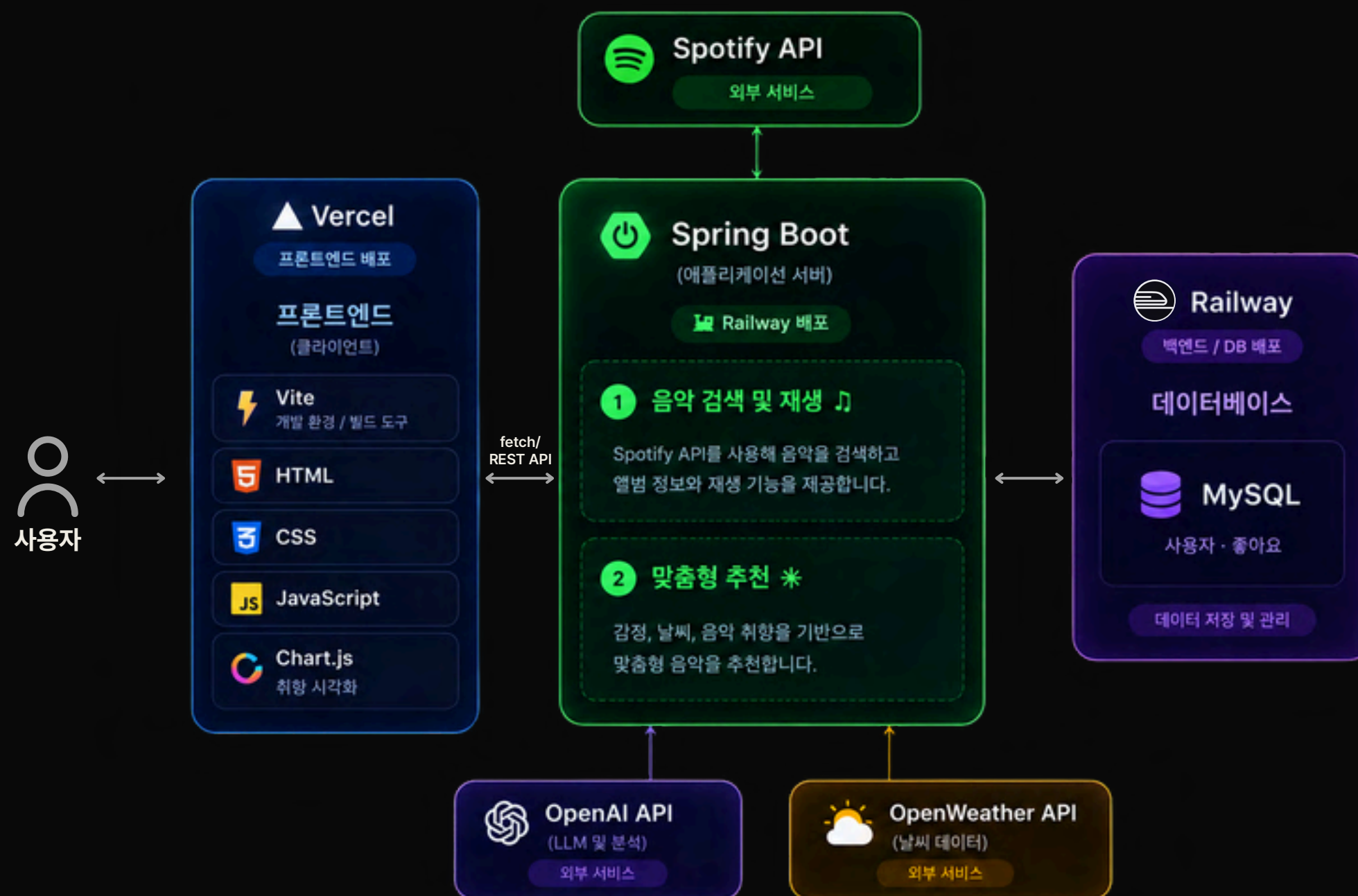
Deployment



Vercel



Railway





MOODWAVE

src

├─ assets

│ └─ css

│ └─ components

│ └─ pages

│ └─ setting

└─ js

└─ api

└─ components

└─ header.js

└─ sidebar.js

└─ footer.js

└─ loading.js

└─ songTable.js

└─ pages

└─ home.js

└─ emotion.js

└─ ...

└─ utils

└─ auth.js

└─ toast.js

└─ ...

└─ data.js

└─ main.js

components

➤ 공통 UI를 **컴포넌트 단위**로 분리

Header, Sidebar, Footer 모든 페이지에서 재사용

Loading, SongTable 독립 컴포넌트로 관리

pages

➤ 페이지별 로직을 분리하여 **기능별 책임** 분리

페이지 수정 시 다른 기능에 영향 최소화

utils

➤ 공통 기능을 분리하여 여러 페이지에서 **재사용**

인증 · 토스트 메시지 · 플레이리스트 저장 등

// 코드 재사용성과 유지보수성 확보



// main.js

```
main.js
1  const routes = [
2    {
3      path: "#/search",
4      protected: true,
5      render: renderSearch,
6      init: initSearch,
7    },
8    {
9      path: "#/latest",
10     protected: true,
11     render: renderLatestPage,
12     init: initLatestPage,
13   },
14   ...
15 ];
16
17 ...
18
19 ...
20
21 function findRoute(hash) {
22   return routes.find((route) => hash.startsWith(route.path));
23 }
24
25 ...
26
27 async function renderPage(main, route) {
28   main.className = "main";
29
30   if (route.pageClass) {
31     main.classList.add(route.pageClass);
32   }
33
34   main.innerHTML = route.render();
35
36   if (route.init) {
37     await route.init();
38   }
39 }
```

```
main.js
1  async function router() {
2    const main = document.querySelector("#main");
3    const hash = location.hash || "#/home";
4
5    if (!main) return;
6
7    const route = findRoute(hash);
8
9    if (route?.protected) {
10     const loggedIn = await isLoggedIn();
11
12     if (!loggedIn) {
13       alert("로그인 후 이용할 수 있습니다.");
14       location.hash = "#/home";
15       return;
16     }
17   }
18
19   if (route) {
20     await renderPage(main, route);
21     return;
22   }
23
24   await renderPage(main, {
25     render: renderHome,
26     init: initHome,
27   });
28 }
29
30 ...
31
32
```

```
main.js
1
2  ...
3
4  function init() {
5    const sidebar = document.querySelector("#sidebar");
6    const header = document.querySelector("#header");
7    const footer = document.querySelector("#footer");
8
9    sidebar.innerHTML = renderSidebar();
10   header.innerHTML = renderHeader();
11   footer.innerHTML = renderFooter();
12
13   initSidebar();
14   initHeader();
15   initFooter();
16
17   router();
18
19   window.addEventListener("hashchange", router);
20 }
```




핵심 기능 소개

moodwave

로그인 연동

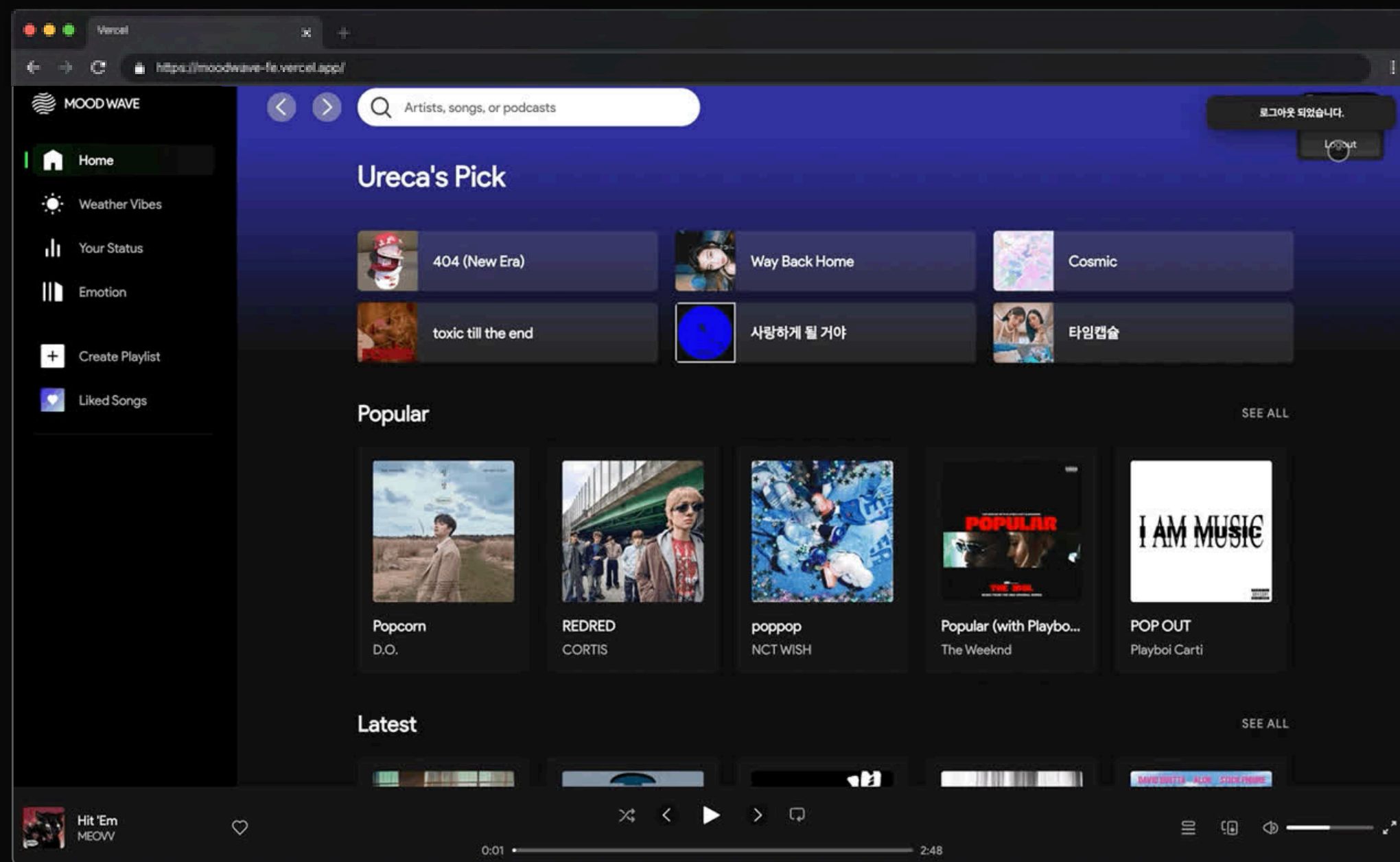
LOGIN

로그인 연동

SPOTIFY 계정 연동을 통해 간편하게 로그인할 수 있으며, 사용자 인증 정보를 기반으로 개인화된 음악 서비스를 제공합니다.



사용자 인증
Spotify & OAuth

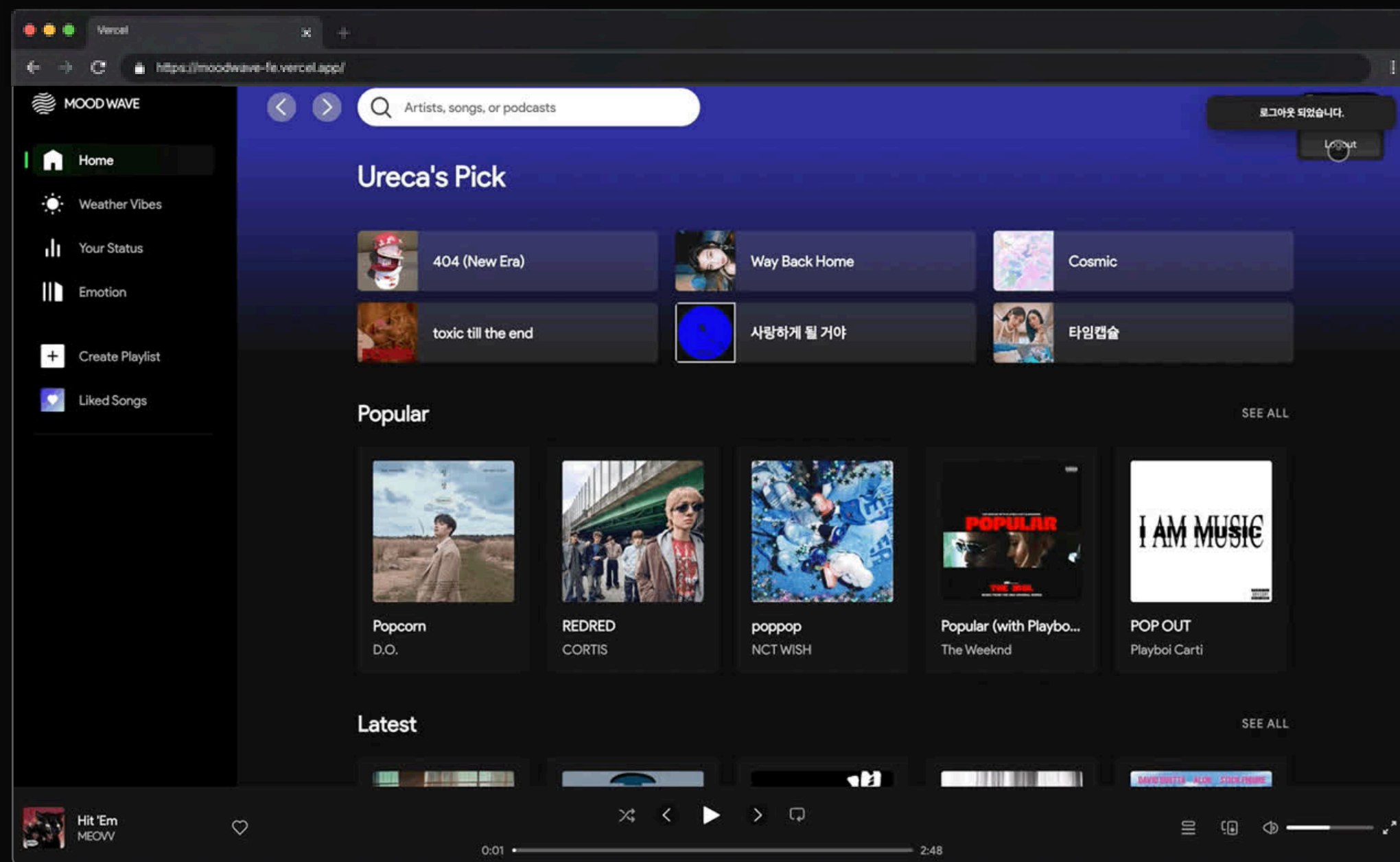




➤ Spotify API를 통해 사용자의 Spotify Id · 이름 · 이메일 을 가져옴

➤ OAuth를 통해 안전하게 사용자의 Spotify 정보를 가져오고 토큰을 통해 사용자의 세션을 관리

➤ Access Token을 이벤트 동작 시 백엔드에서 검사하여 토큰 만료여부를 확인, 만료 시 로그인 / 만료 전에는 토큰 재발급





// SecurityConfig.java (Backend)

```
SecurityConfig.java
1 .oauth2Login(oauth -> oauth
2   .userInfoEndpoint(userInfo -> userInfo
3     .userService(customOAuth2UserService)
4   )
5   .successHandler((request, response, authentication) -> {
6     String redirectUrl = getRedirectUrlFromSession(request);
7
8     if (!isAllowedRedirect(redirectUrl)) {
9       redirectUrl = DEFAULT_FRONT_URL;
10    }
11
12    response.sendRedirect(redirectUrl);
13  })
14 )
```

// header.js (Frontend)

```
header.js
1 function getSpotifyLoginUrl() {
2   const redirectUrl = window.location.href;
3
4   return `${API_ENDPOINTS.spotifyLogin}?redirect=${encodeURIComponent(redirectUrl)}`;
5 }
6
7 spotifyConnectBtn.addEventListener("click", () => {
8   spotifyConnectBtn.disabled = true;
9   window.location.href = getSpotifyLoginUrl();
10 });
```

// CustomOAuth2UserService.java (Backend)

```
CustomOAuth2UserService.java
1 OAuth2User oAuth2User = super.loadUser(userRequest);
2
3 Map<String, Object> attributes = oAuth2User.getAttributes();
4 String spotifyId = (String) attributes.get("id");
5 String name = (String) attributes.get("display_name");
6 String email = (String) attributes.get("email");
7
8 saveOrUpdate(spotifyId, name, email);
```



핵심 기능 소개

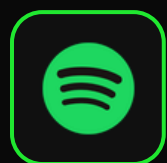
moodwave

검색 및 재생

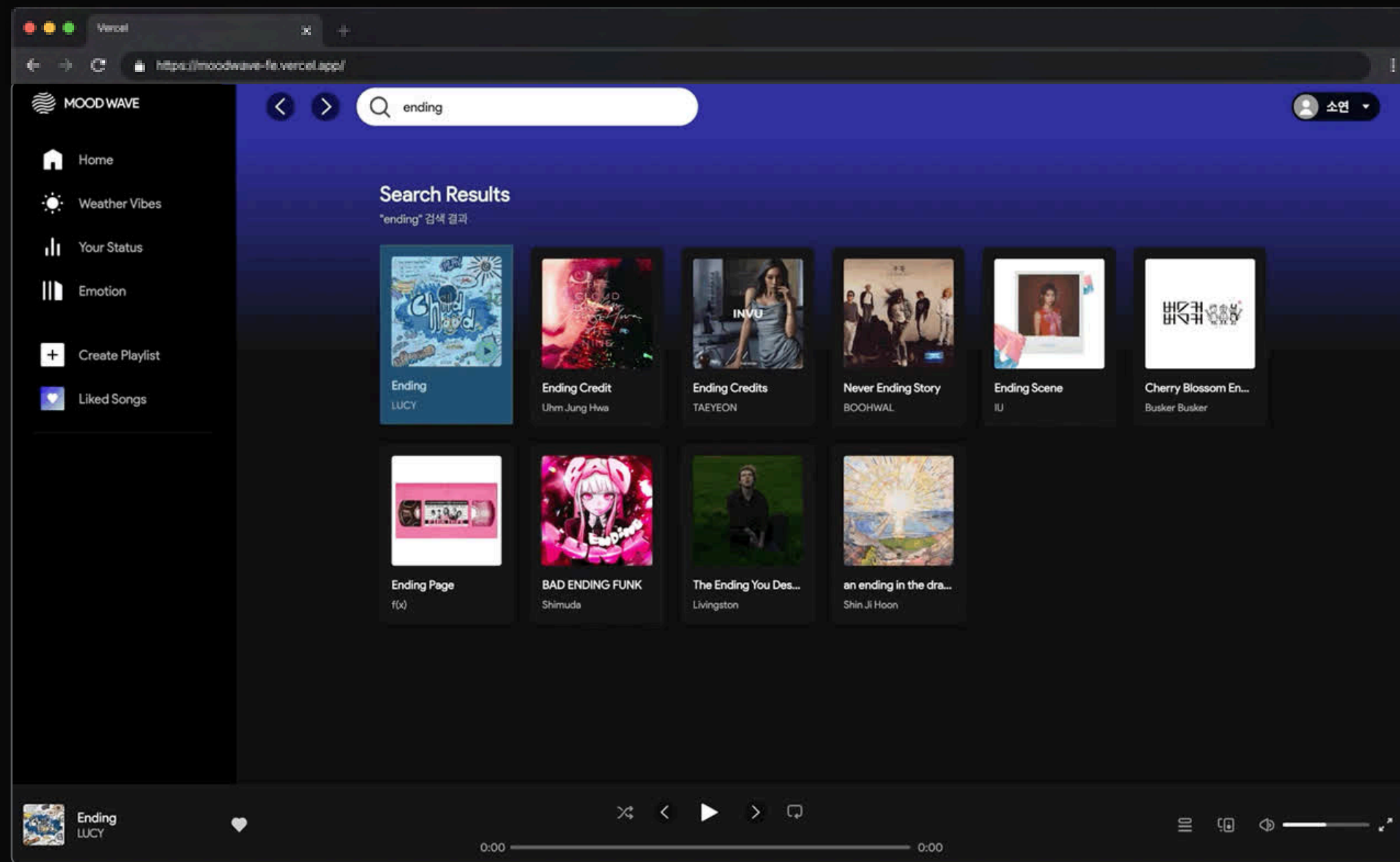
SEARCH / PLAY

검색 및 재생

SPOTIFY API를 활용해 원하는 음악을 검색하고
선택한 곡을 즉시 재생할 수 있는 기능입니다.

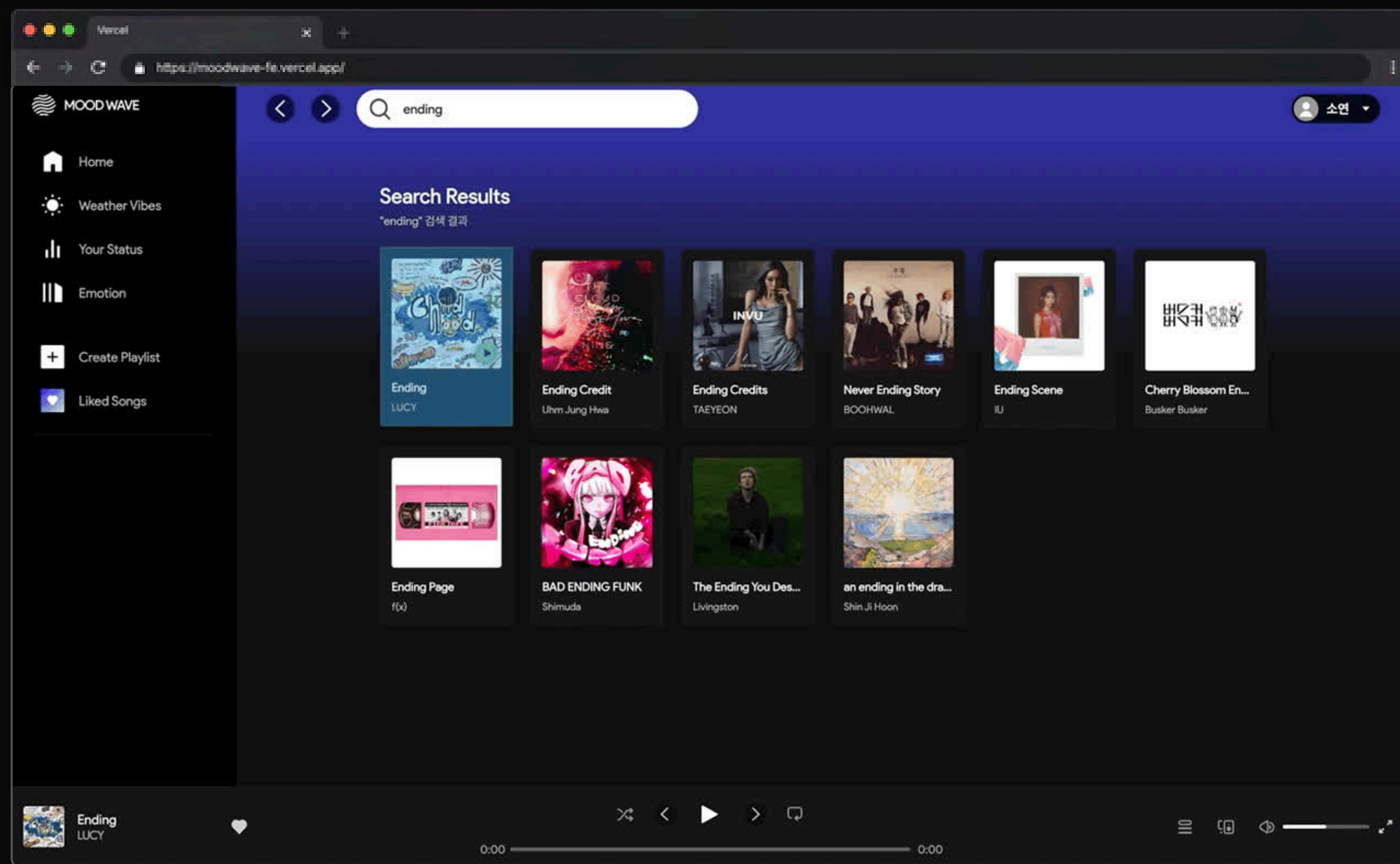


SPOTIFY 연동
Spotify API





- Spotify API를 활용해 사용자가 입력한 키워드와 관련된 곡 · 아티스트를 검색
- 검색 결과를 카드와 테이블 형태로 제공해 원하는 음악을 쉽게 탐색
- Footer Player와 연동해 선택한 곡을 하단 플레이어에서 바로 재생
- 재생·일시정지·진행바·볼륨 조절을 지원해 실제 음악 앱처럼 컨트롤 가능





// Search.js (Frontend)

```
search.js

1 async function fetchSearchTracks(keyword) {
2   const response = await fetch(
3     `${HOME_API_URL}/search?keyword=${encodeURIComponent(keyword)}`,
4   );
5
6   if (!response.ok) {
7     throw new Error("SEARCH_API_ERROR");
8   }
9
10  return response.json();
11 }
12
13 export async function initSearch() {
14   const keyword = getSearchKeyword();
15
16   if (!keyword) {
17     renderSearchResult([]);
18     return;
19   }
20
21   renderSearchMessage("검색 중...", "search-page__loading");
22
23   try {
24     const tracks = await fetchSearchTracks(keyword);
25     renderSearchResult(tracks);
26   } catch (error) {
27     renderSearchMessage("검색 결과를 불러오지 못했습니다.");
28   }
29 }
```

// HomeController.java (Backend)

```
HomeController.java

1 @GetMapping("/home/search")
2 public List<Map<String, Object>> searchTracks(
3     @RequestParam(value = "keyword", required = false) String keyword
4 ) {
5     if (keyword == null || keyword.trim().isEmpty()) {
6         return List.of();
7     }
8
9     return spotifyService.searchTracksByKeyword(keyword.trim(), 50);
10 }
```

// SpotifyService.java (Backend)

```
SpotifyService.java

1 public List<Map<String, Object>> searchTracksByKeyword(String keyword, int displayLimit) {
2
3     String trimmedKeyword = keyword == null ? "" : keyword.trim();
4
5     if (trimmedKeyword.isEmpty()) {
6         return List.of();
7     }
8
9     int safeDisplayLimit = Math.min(Math.max(displayLimit, 1), 10);
10
11     String accessToken = getAccessToken();
12
13     List<Map<String, Object>> result =
14         searchTracksByKeywordFallback(trimmedKeyword, safeDisplayLimit, accessToken);
15
16     return result;
17 }
```



// footer.js

```
1 spotifyPlayer = new window.Spotify.Player({
2   name: "MOOD WAVE Player",
3
4   getOAuthToken: async (callback) => {
5     const token = spotifyAccessToken || (await getSpotifyAccessToken());
6     callback(token);
7   },
8
9   volume: 0.5,
10 });
11
12 ...
13
14 spotifyPlayer.addListener("ready", async ({ device_id }) => {
15   spotifyDeviceId = device_id;
16   isSpotifyReady = true;
17
18   await transferPlaybackToMoodWave();
19
20   currentVolume = await spotifyPlayer.getVolume();
21   previousVolume = currentVolume || 0.5;
22   updateVolumeUI();
23 });
```

```
1 playButton.addEventListener("click", async () => {
2   if (!checkSpotifyPlayerReady()) return;
3
4   if (!hasStartedPlayback) {
5     await playTestTrack();
6     return;
7   }
8
9   await spotifyPlayer.togglePlay();
10 });
11
12 prevButton.addEventListener("click", async () => {
13   if (!checkSpotifyPlayerReady()) return;
14
15   unlockTrackUIByCardClick();
16   resetProgressForTrackChange();
17
18   await spotifyPlayer.previousTrack();
19 });
20
21 ...
```

```
1 spotifyPlayer.addListener("player_state_changed", (state) => {
2   if (!state) return;
3
4   const currentTrack = state.track_window.current_track;
5
6   if (!currentTrack) return;
7
8   currentTrackId = currentTrack.id;
9
10  currentTrackInfo = {
11    musicId: currentTrack.id,
12    title: currentTrack.name,
13    artist: currentTrack.artists?.map((a) => a.name).join(", "),
14    albumImage: currentTrack.album?.images?.[0]?.url || "",
15    duration: state.duration,
16  };
17
18  isPlaying = !state.paused;
19  updatePlayButtonIcon();
20
21  if (!isTrackUiLockedByCardClick) {
22    updateCurrentTrackUI(currentTrack);
23  }
24
25  updateProgressUI(state.position, state.duration);
26 });
```




핵심 기능 소개

moodwave

좋아요 & 플레이리스트

LIKED

좋아요

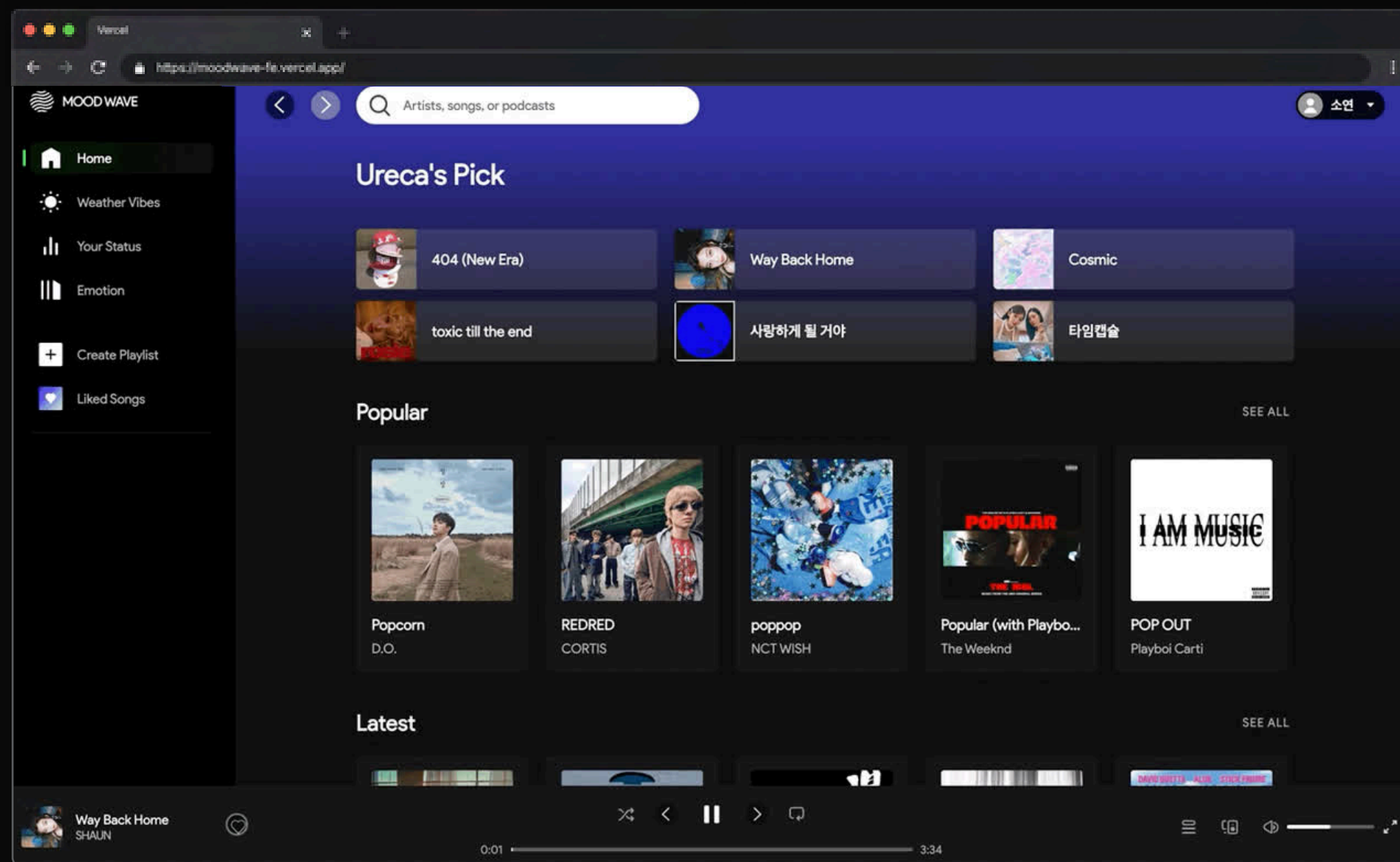
플레이리스트

마음에 드는 곡을 좋아요 목록에 저장하고,
나만의 플레이리스트를 생성 및 관리할 수 있습니다.



좋아요 & 플레이리스트 관리

Spring Boot & MySQL

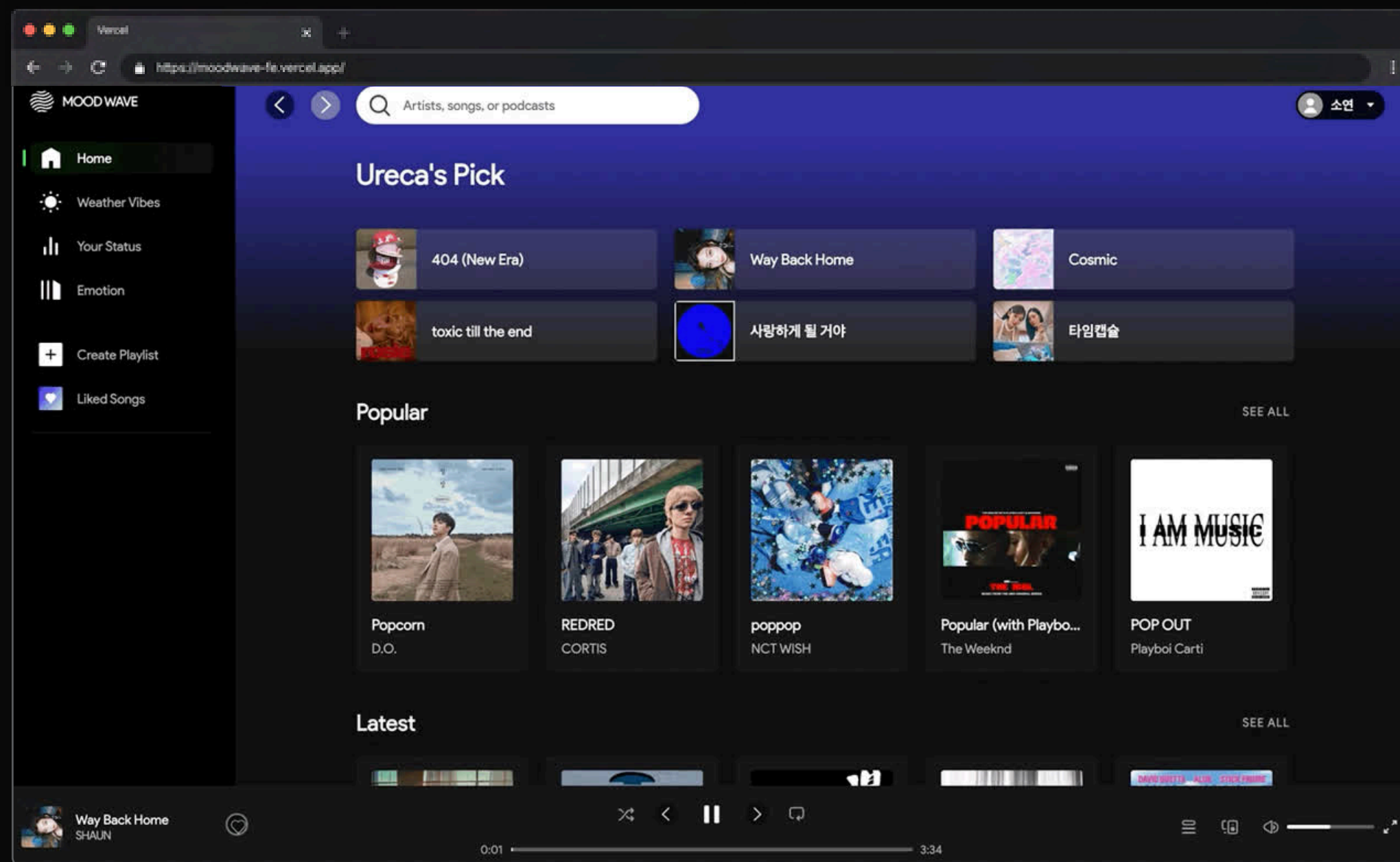




➤ 좋아요 버튼 클릭 시 아이콘 변하고 DB에 저장

➤ 좋아요 목록 클릭 시 DB에 저장된 좋아요 불러오기

➤ 좋아요 목록 에서 좋아요 아이콘 클릭시 DB에서 삭제 및 좋아요 목록 재렌더링





// footer.js (Frontend)

```
1 likeButton.addEventListener("click", async () => {
2   if (!currentTrackId) {
3     alert("재생 중인 곡이 없습니다.");
4     return;
5   }
6
7   const response = await fetch(LIKE_API_URL, {
8     method: "POST",
9     headers: {
10       "Content-Type": "application/json",
11     },
12     body: JSON.stringify(currentTrackInfo),
13     credentials: "include",
14   });
15
16   if (response.ok) {
17     alert("좋아요가 반영되었습니다.");
18     isLiked();
19     window.dispatchEvent(new Event("likeChanged"));
20   }
21 });
```

// LikeService.java (Backend)

```
1 public boolean add(LikeVO vo) {
2   LikeVO existing = likeMapper.isLike(vo.getSpotifyId(), vo.getMusicId());
3
4   if (existing == null) {
5     return likeMapper.insertLike(vo) > 0;
6   }
7
8   return false;
9 }
10
11 public boolean remove(String spotifyId, String musicId) {
12   return likeMapper.deleteLike(spotifyId, musicId) > 0;
13 }
```



핵심 기능 소개

moodwave

날씨 기반 추천 플레이리스트

WEATHER VIBES

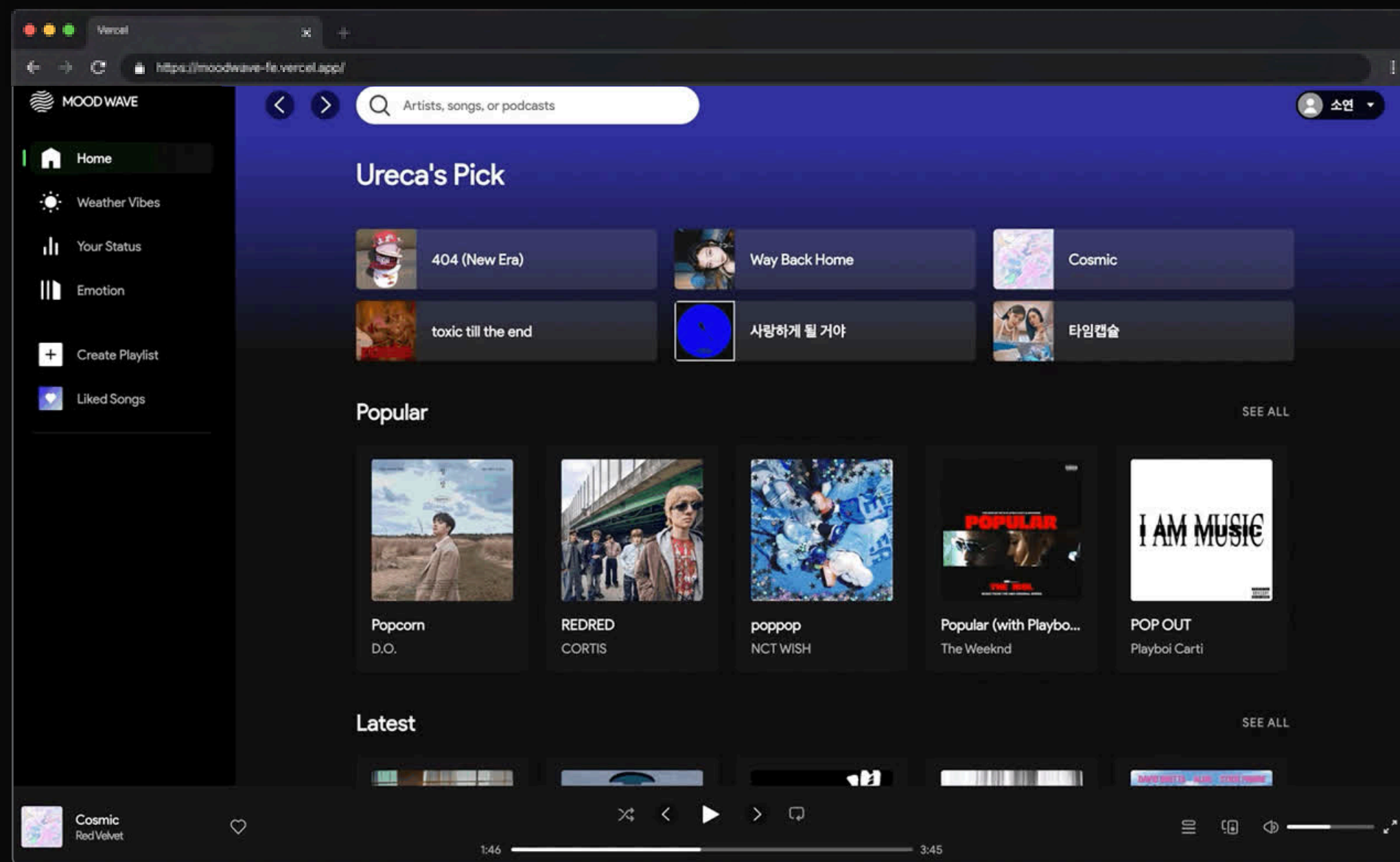
날씨 기반 플레이리스트

현재 위치의 실시간 날씨 정보를 기반으로
날씨 분위기에 어울리는 플레이리스트를
추천합니다.



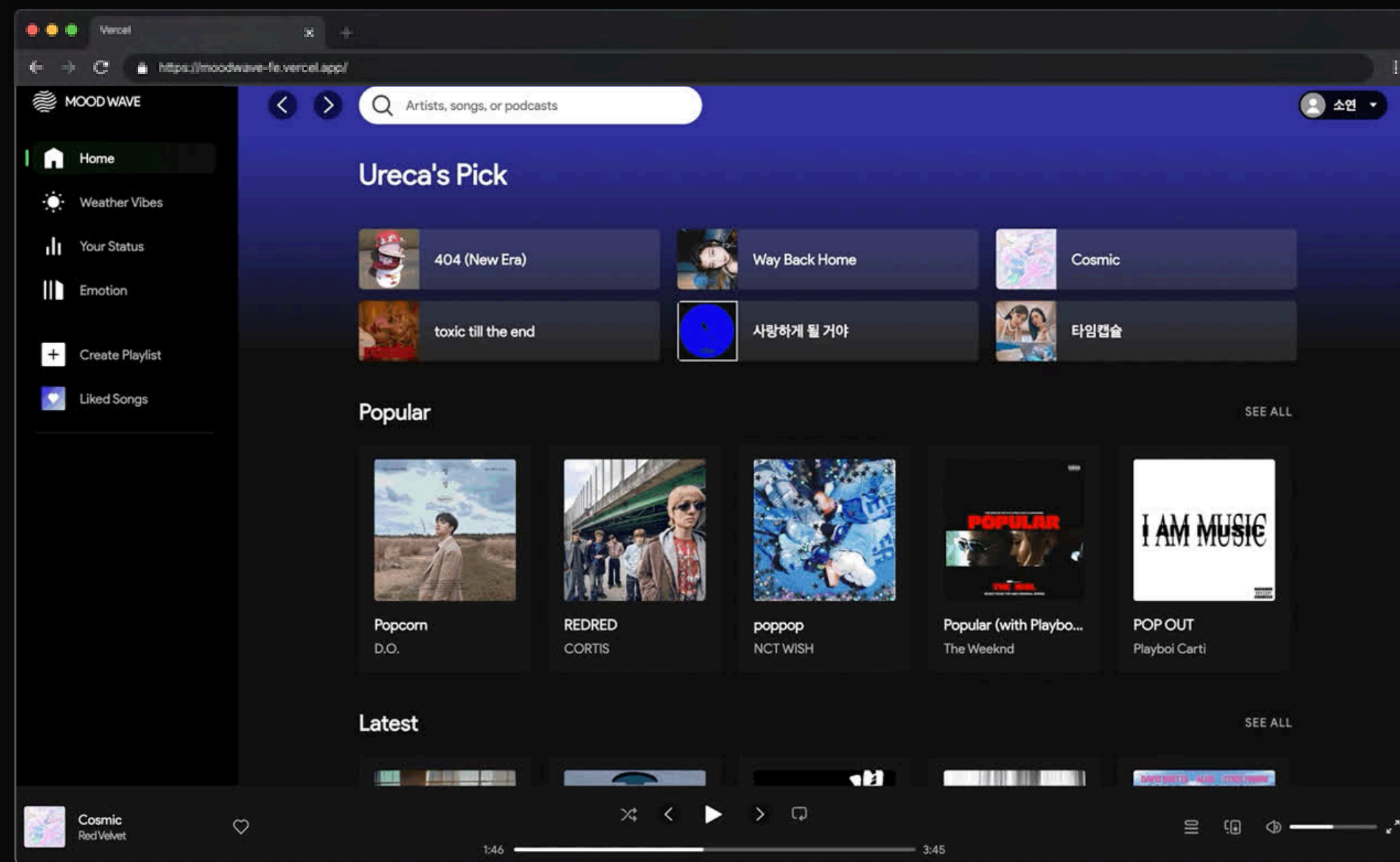
현재 위치 기반 날씨 조회

OpenWeather API





- OpenWeather API를 통해 현재 위치 기반 실시간 날씨 정보를 조회
- 세분화된 날씨 코드를 6개의 날씨 카테고리로 분류하여 플레이리스트와 매핑
- 현재 날씨에 맞는 대표 플레이리스트를 자동 추천
- 다른 날씨의 플레이리스트도 함께 탐색하도록 구성





// weather.js

```
1 function normalizeWeather(weather) {
2   if (
3     weather === "Mist" ||
4     weather === "Fog" ||
5     weather === "Haze" ||
6     weather === "Smoke" ||
7     weather === "Dust" ||
8     weather === "Sand" ||
9     weather === "Ash" ||
10    weather === "Squall" ||
11    weather === "Tornado"
12  ) {
13    return "Foggy";
14  }
15
16  return weather;
17 }
18
```

```
1 async function getWeather(lat, lon) {
2   if (!WEATHER_API_KEY) {
3     console.error("VITE_WEATHER_API_KEY가 설정되지 않았습니다.");
4     setLoading("weatherLoading", false);
5     return;
6   }
7
8   const url = `https://api.openweathermap.org/data/2.5/weather?
lat=${lat}&lon=${lon}&appid=${WEATHER_API_KEY}&units=metric`;
9
10  try {
11    const response = await fetch(url);
12
13    if (!response.ok) {
14      throw new Error("날씨 정보를 불러오지 못했습니다.");
15    }
16
```

```
17   const data = await response.json();
18
19   const weather = normalizeWeather(data.weather[0].main);
20   const temp = Math.round(data.main.temp);
21   const city = data.name;
22
23   weatherInfo.textContent = `${temp}°C · ${city}`;
24
25   updateWeather(weather);
26   updatePlaylist(weather);
27   renderWeatherCards(weather);
28
29   setWeatherPageLoaded(true);
30 } catch (error) {
31   console.error(error);
32   setLoading("weatherLoading", false);
33 }
34 }
```

// weather-playlist.js

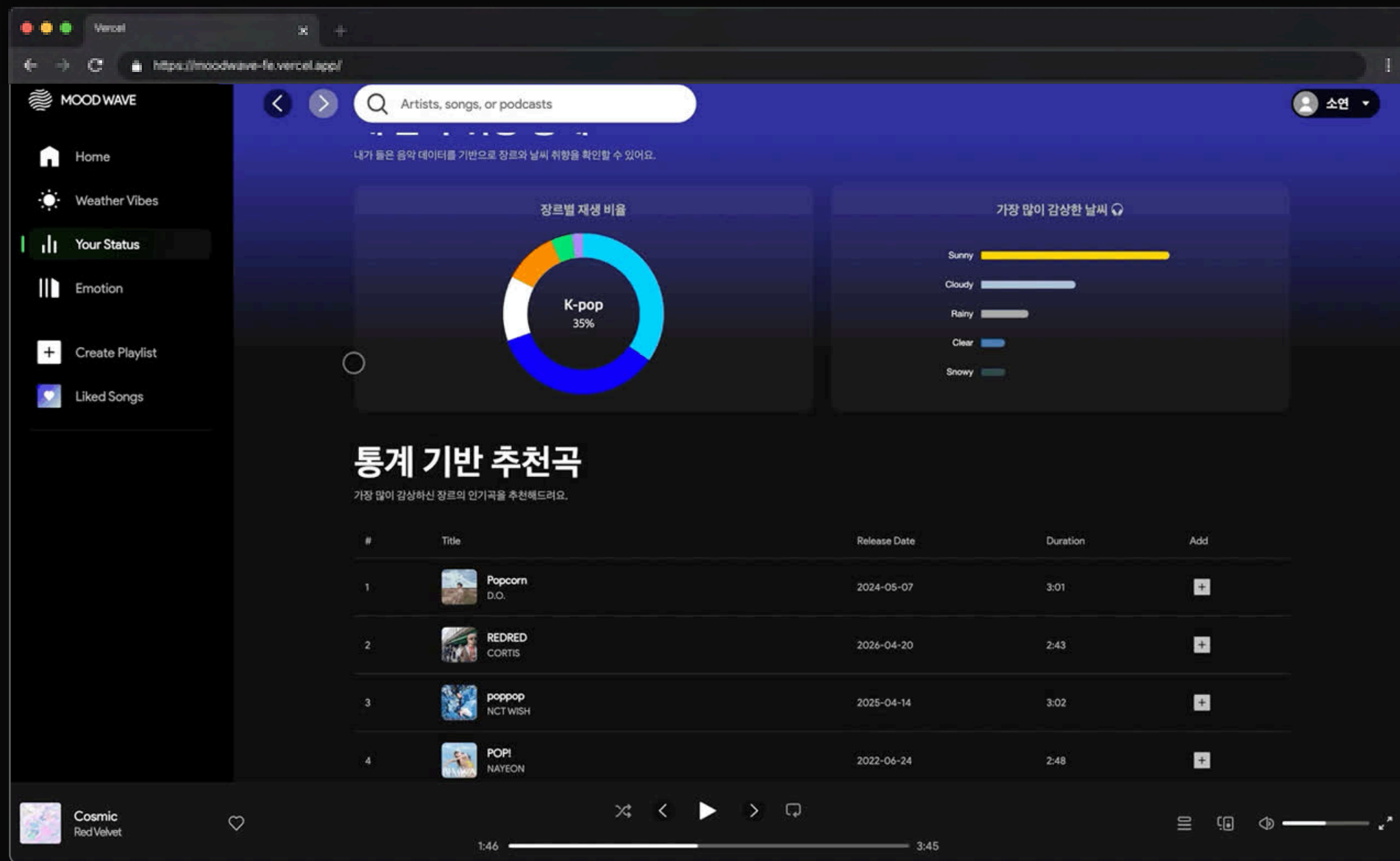
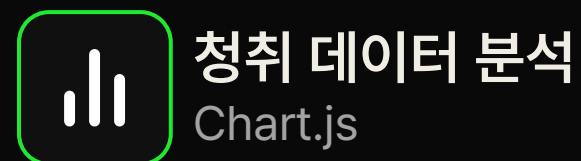
```
1 // =====
2 // 예: #/weather-playlist?playlist=Rain
3 // =====
4 function getPlaylistTypeFromHash() {
5   const queryString = location.hash.split("?")[1];
6   const params = new URLSearchParams(queryString);
7
8   const playlistType = params.get("playlist");
9
10  if (playlistMap[playlistType]) {
11    return playlistType;
12  }
13
14  return "Rain";
15 }
```



YOUR STATUS

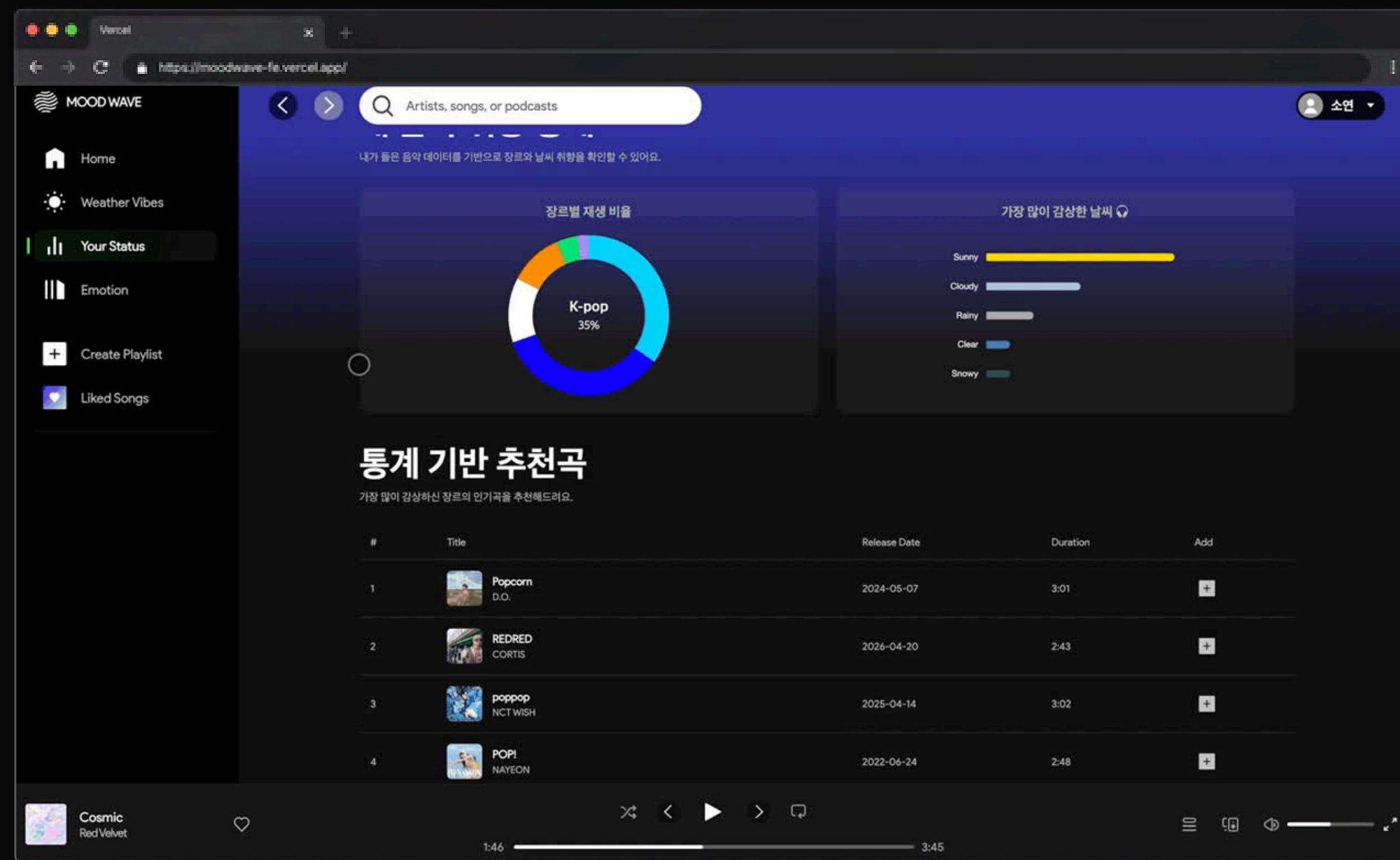
음악 취향 통계

사용자의 장르별 재생 비율과 가장 많이 감상한 날씨 플레이리스트 곡들의 데이터를 시각화하여 음악 취향을 한눈에 확인할 수 있습니다.





- Chart.js를 활용해 장르별 재생 비율과 선호 날씨를 시각화
- 사용자가 가장 많이 들은 장르의 인기곡을 추천
- 아티스트 매핑 테이블과 곡 ID 기반 해시 매핑으로 장르·날씨를 분류





// Chart.js

```
chart.js
1 function getTopDataByField(dataArray, field) {
2   const stats = dataArray.reduce((acc, item) => {
3     const label = normalizeText(item?.[field], "Unknown");
4
5     acc[label] = (acc[label] || 0) + 1;
6     return acc;
7   }, {});
8
9   return Object.entries(stats)
10     .sort((a, b) => b[1] - a[1])
11     .slice(0, 6);
12 }
```

// Chart.js

```
chart.js
1 function renderGenreChart(topGenres) {
2   const genreCanvas = document.querySelector("#genreChart");
3
4   if (!genreCanvas || typeof Chart === "undefined") {
5     return;
6   }
7
8   const genreCtx = genreCanvas.getContext("2d");
9
10  new Chart(genreCtx, {
11    type: "doughnut",
12    data: {
13      labels: topGenres.map(([label]) => label),
14      datasets: [
15        {
16          data: topGenres.map([, value]) => value,
17        },
18      ],
19    },
20    options: {
21      plugins: {
22        legend: { display: false },
23        tooltip: { enabled: true },
24      },
25      cutout: "70%",
26    },
27  });
28 }
```




YOUR STATUS

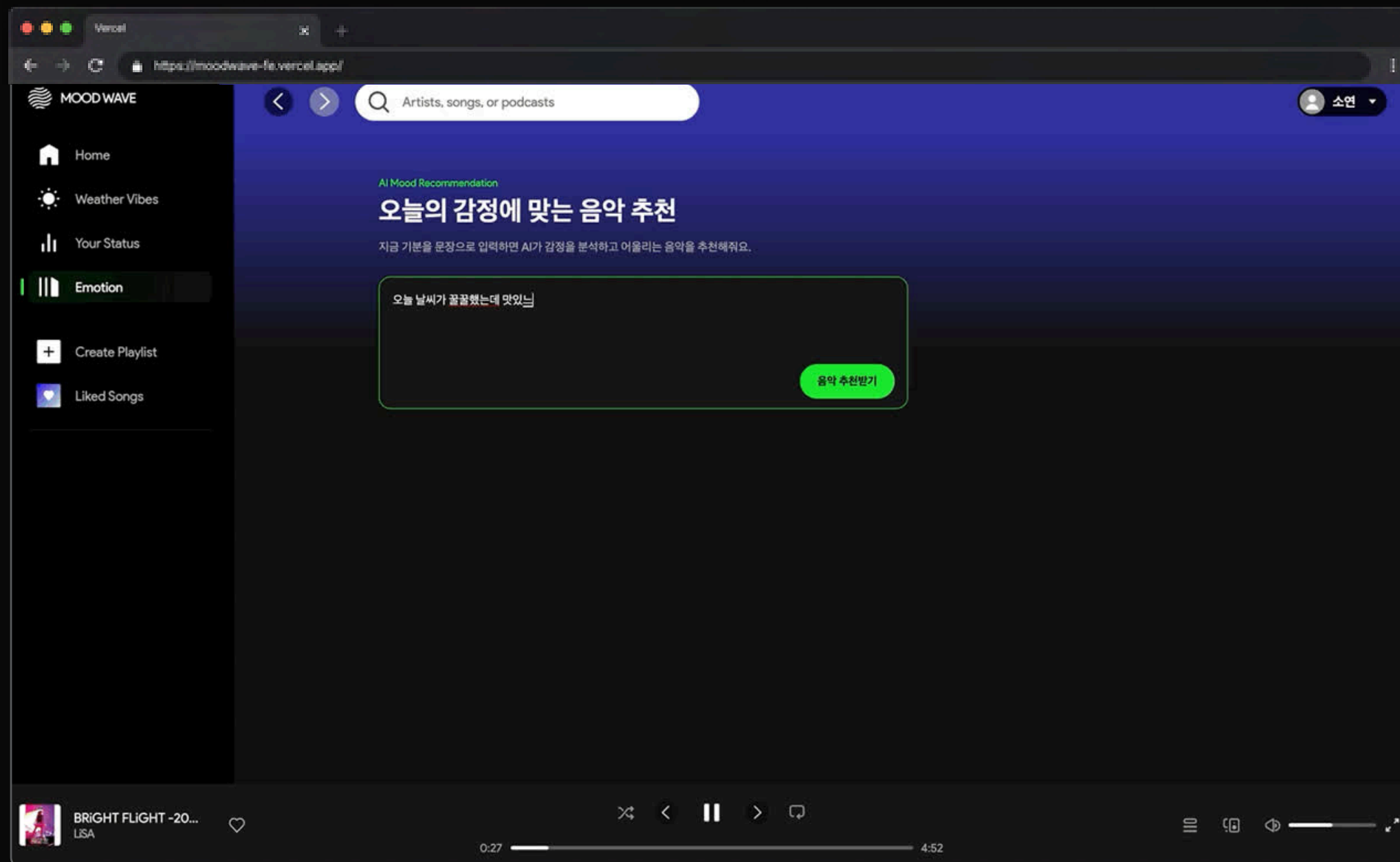
AI 감정 분석 플레이리스트

사용자가 입력한 감정을 AI가 분석하여
현재 감정에 어울리는 음악을 추천합니다.



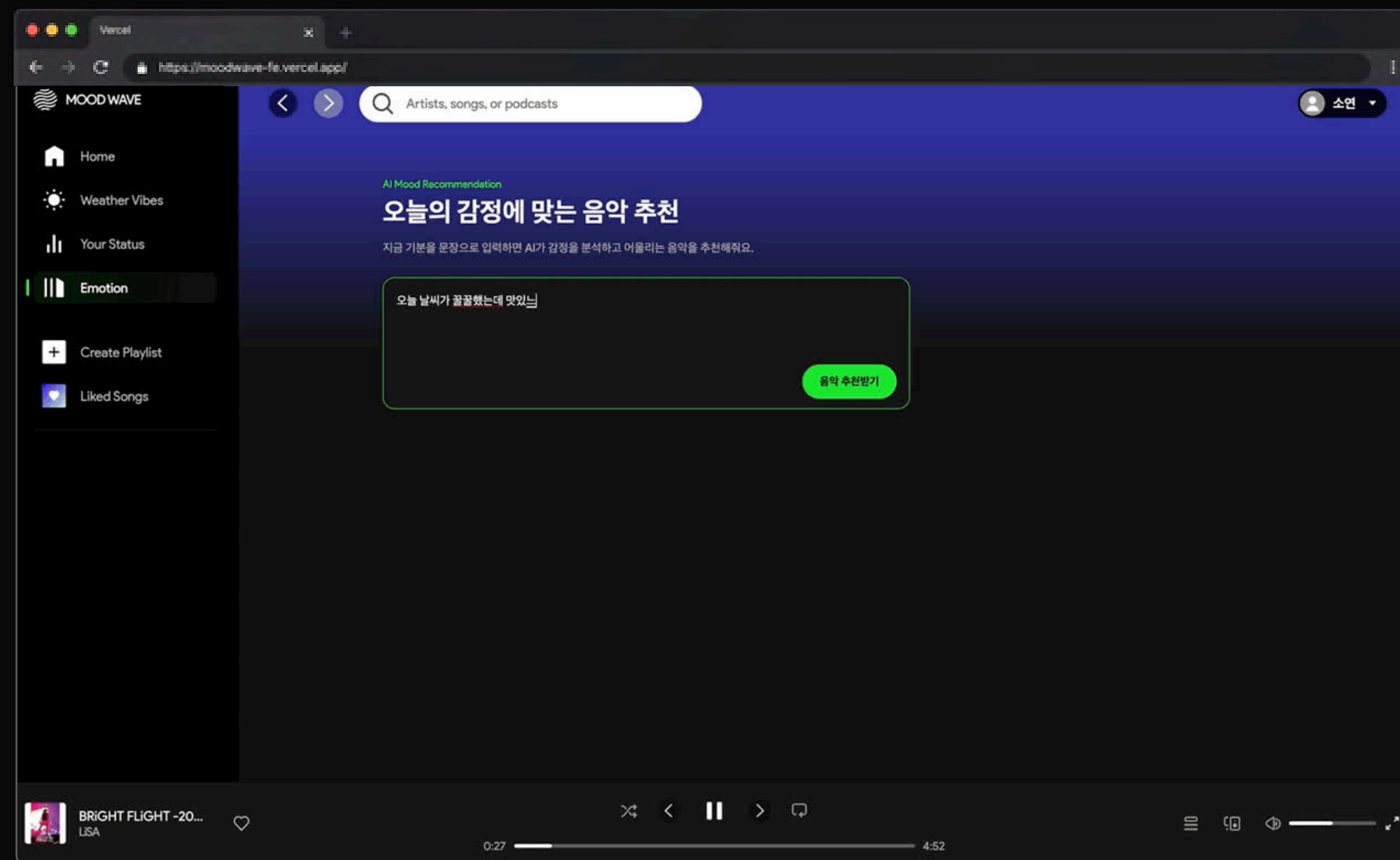
AI 감정 분석

Gemini & Spotify API





- OpenAI API를 활용해 사용자가 입력한 문장의 감정과 추천 의도를 분석
- 감정과 의도를 분리해 위로, 공감, 전환, 몰입 등 추천 방향을 결정
- 분석 결과를 기반으로 Spotify 검색 키워드를 생성해 감정에 맞는 곡을 조회
- 같은 감정이라도 사용자의 표현과 요청에 따라 다른 분위기의 음악을 추천





// EmotionRecommendService.java (Backend)

```
EmotionRecommendService.java

1 EmotionAnalyzeResponse analysis = openAiEmotionService.analyzeEmotion(userText);
2
3 String emotion = normalizeValue(analysis.emotion(), "NEUTRAL");
4 String intent = normalizeValue(analysis.recommendationIntent(), "NEUTRAL");
5
6 intent = overrideIntentByUserText(userText, intent);
7
8 ...
9
10 private String overrideIntentByUserText(String text, String currentIntent) {
11     String normalizedText = normalizeText(text);
12
13     if (containsAny(normalizedText, List.of(
14         "신나는",
15         "신나",
16         "기분전환",
17         "텐션",
18         "밝은",
19         "댄스"
20     ))) {
21         return "UPLIFTING";
22     }
23
24     if (containsAny(normalizedText, List.of(
25         "위로",
26         "위로해",
27         "힘들어",
28         "따뜻한",
29         "힐링"
30     ))) {
31         return "COMFORT";
32     }
33
34     ...
35
36     return currentIntent;
37 }
```

// OpenAiEmotionService.java (Backend)

```
EmotionRecommendService.java

1 """
2 너는 음악 추천 서비스 MOOD WAVE의 감정 분석기다.
3
4 사용자의 문장을 분석해서 반드시 아래 두 가지를 분리해라.
5
6 1. emotion
7 - 사용자가 현재 느끼는 감정이다.
8 - 슬픔, 분노, 불안, 피곤함, 행복, 설렘 등을 판단한다.
9
10 2. recommendationIntent
11 - 사용자가 원하는 음악 추천 방향이다.
12 - 현재 감정과 추천 방향은 다를 수 있다.
13
14 예시:
15 - "나 너무 슬퍼"
16   => emotion: SAD
17   => recommendationIntent: MATCH_MOOD
18
19 - "나 너무 슬픈데 신나는 노래 추천해줘"
20   => emotion: SAD
21   => recommendationIntent: UPLIFTING
22
23 ...
24
25 판단 규칙:
26 - 감정 단어만 보고 추천 방향을 고정하지 마라.
27 - "신나는", "기분전환", "텐션", "밝은", "업되는" 표현이 있으면 UPLIFTING으로 판단해라.
28 - "더 끌어올릴", "폭발", "강렬한", "빽센", "에너지" 표현이 있으면 ENERGETIC으로 판단해라.
29
30 ...
31
32 """
```



// SpotifyService.java (Backend)

```
SpotifyService.java
1 public List<Map<String, Object>> searchTracksForMood(String keyword, int displayLimit) {
2     sortedTracks.sort((a, b) -> {
3         int scoreA = calculateMoodSearchScore(a, trimmedKeyword);
4         int scoreB = calculateMoodSearchScore(b, trimmedKeyword);
5
6         return Integer.compare(scoreB, scoreA);
7     });
8 }
9
10 ...
11
12 private int calculateMoodSearchScore(Map<String, Object> track, String keyword) {
13     int score = 0;
14
15     for (String word : words) {
16         if (title.contains(word)) {
17             score += 300;
18         }
19
20         if (artistText.contains(word)) {
21             score += 150;
22         }
23
24         if (albumName.contains(word)) {
25             score += 120;
26         }
27     }
28
29     score += popularityScore * 3;
30
31     if (containsMoodBadVersionWord(title)) {
32         score -= 1200;
33     }
34
35     if (containsMoodBadVersionWord(albumName)) {
36         score -= 700;
37     }
38
39     return score;
40 }
```

// emotion.js (Frontend)

```
emotion.js
1 async function requestEmotionRecommend(text) {
2     const response = await fetch(API_ENDPOINTS.emotionRecommend, {
3         method: "POST",
4         headers: {
5             "Content-Type": "application/json",
6         },
7         body: JSON.stringify({ text }),
8     });
9
10    if (!response.ok) {
11        throw new Error("감정 추천 API 요청 실패");
12    }
13
14    return response.json();
15 }
```




기 박해준

- MOOD WAVE 프로젝트를 통해 화면 구현뿐만 아니라 사용자 흐름, API 연동, 로그인 상태 관리, OAuth/CORS/배포 환경 문제 해결까지 경험하며 실무적인 개발 과정을 배울 수 있었다.
- 아쉬웠던 더미 데이터와 로컬스토리지 기반 기능은 향후 DB 저장 방식으로 개선하여 재생 기록 기반 개인 맞춤형 음악 추천 기능으로 확장해보고 싶다.

기 송동현

- OAuth와 API 연동을 통해 프론트엔드와 백엔드가 데이터를 주고받는 흐름을 이해했고, 토큰 기반 인증과 SPA 구조, JS 모듈화, 커스텀 이벤트 사용 방법을 배울 수 있었다.
- 또한 협업 과정에서 GitHub 관리와 팀원 간 소통의 중요성을 느꼈으며, 코드 주석도 변수와 함수의 역할을 공유하는 중요한 소통 수단이라는 점을 깨달았다.

기 박소연

- OpenWeather API를 활용한 날씨 기반 음악 추천 기능을 구현하며 API 연동과 데이터 가공 및 화면 렌더링 과정을 경험할 수 있었다.
- 기능별 파일 분리와 공통 컴포넌트 재사용 구조를 적용하면서, 프로젝트 규모가 커지더라도 유지보수와 기능 확장이 용이한 구조 설계의 중요성을 체감했다.

기 이수아

- Chart.js를 활용한 데이터 시각화, 민감 정보 분리 관리, API 연동 과정에서 보안과 환경 설정의 중요성을 배웠고, 기획과 실제 제공 데이터 사이의 차이를 경험할 수 있었다.
- 제한된 기간 안에서 핵심 기능을 우선 구현하며 우선순위 설정의 중요성을 느꼈고, GitHub 협업 컨벤션과 기록, Codex를 활용한 디버깅 경험을 통해 효율적인 협업 방법을 배웠다.



01 날씨 기반 추천 기능 고도화

향후 날씨 데이터를 OpenAI에 전달하여 날씨에 어울리는 음악 키워드와 분위기를 생성하고, 이를 Spotify 음악 데이터와 연동해 더욱 정확한 날씨 맞춤형 음악 추천 기능으로 확장

02 플레이리스트 저장 방식 개선

향후 플레이리스트도 DB에 저장하도록 개선하면 사용자가 다른 기기에서 접속해도 자신의 플레이리스트를 유지할 수 있고, 사용자별 음악 데이터를 더 안정적으로 관리 가능

03 청취 기록 기반 통계 기능 확장

향후 사용자가 재생한 노래의 장르, 아티스트, 재생 횟수 등을 DB에 저장하면 실제 청취 데이터를 기반으로 한 통계 차트를 제공 가능

04 개인 맞춤형 음악 추천 강화

DB에 저장된 사용자의 청취 기록을 분석하여 가장 많이 들은 장르나 선호 아티스트를 파악하고, 해당 장르의 인기 음악 또는 유사한 분위기의 음악을 추천하는 개인화 추천 기능으로 발전

